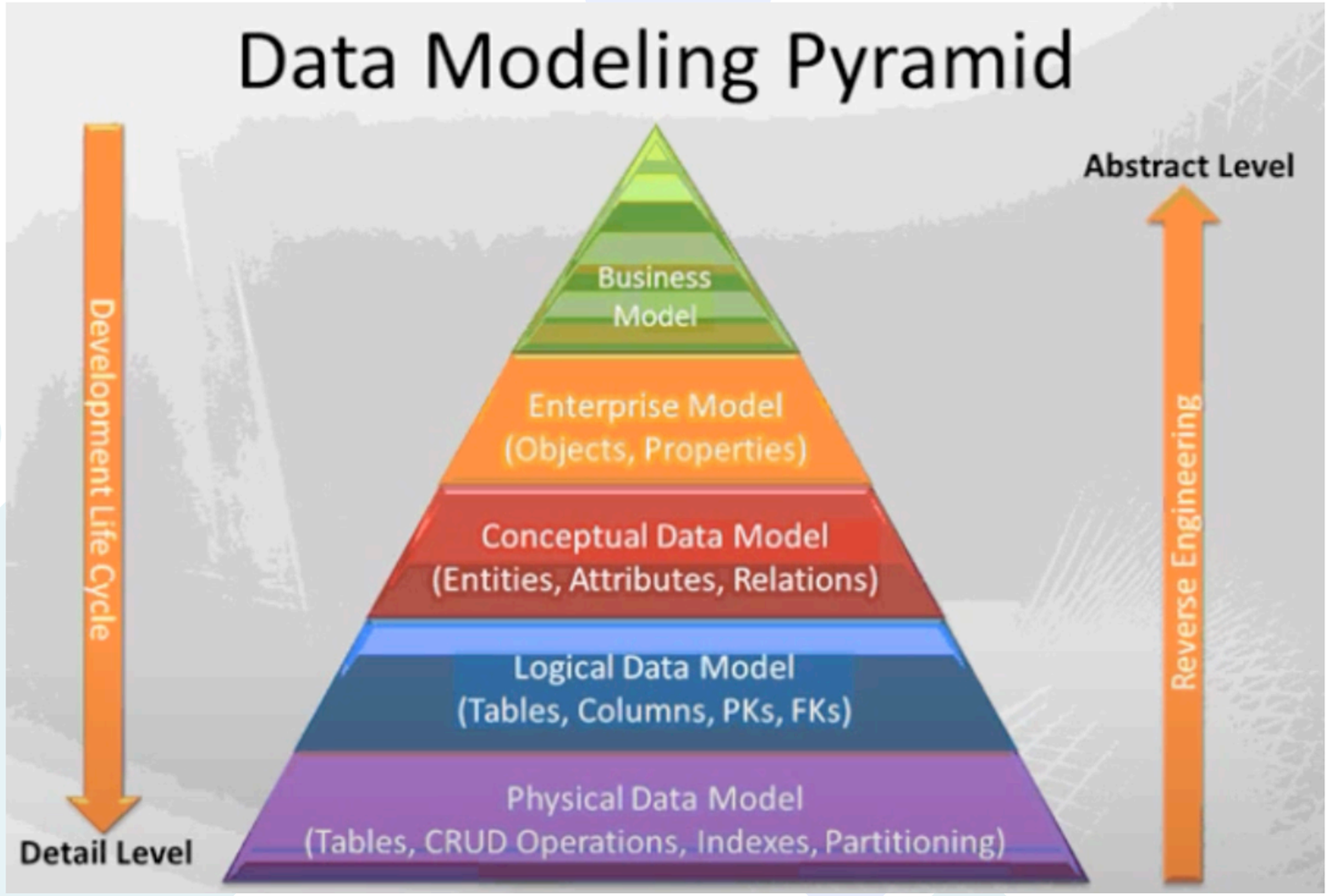
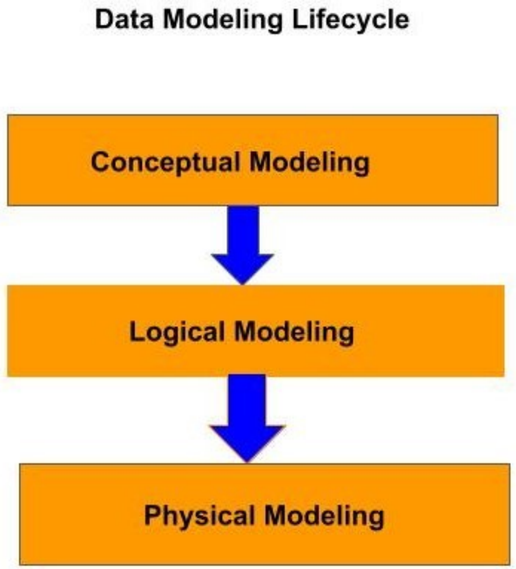
Data Modeling Life Cycle



Data Modeling Life Cycle





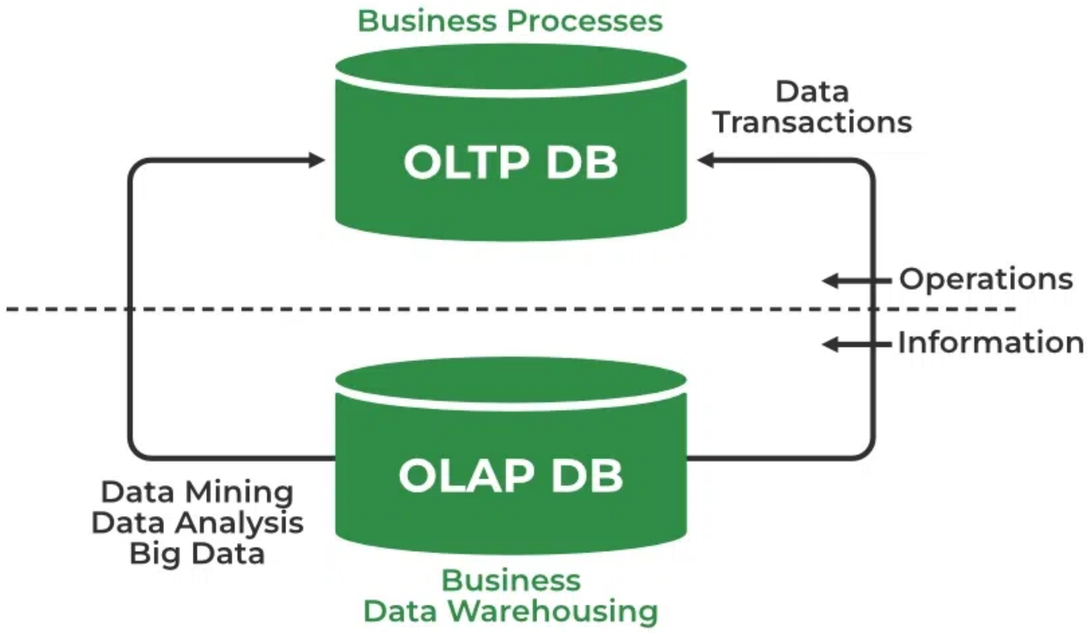
Step 1: Requirement Gathering

- Understand and document the information needs of the system’s users and stakeholders.
  - Interview stakeholders to identify data requirements.
  - Analyze the existing docs and sources
  - Define the scope and boundaries of the data model: budget, timeline etc
- A lot of meetings will be held!
- Product: BRD, JRD, JAD
- Alternatively..
  - Face to face interviews, Anonymous questionnaires, feedback surveys



Step2: Conceptual Data Modeling

- High level representation, without much details(abstract design)
  - Focus on business concepts and requirements
    - What entities exist and how they relate
    - broad picture of the business and its data.
- Product: ER- Diagram
- The decision on whether to use normalized tables (**OLTP**) or denormalized structures (DWH, or **OLAP**) is typically made. Although the level of normalization may be determined in the coming logical phase.

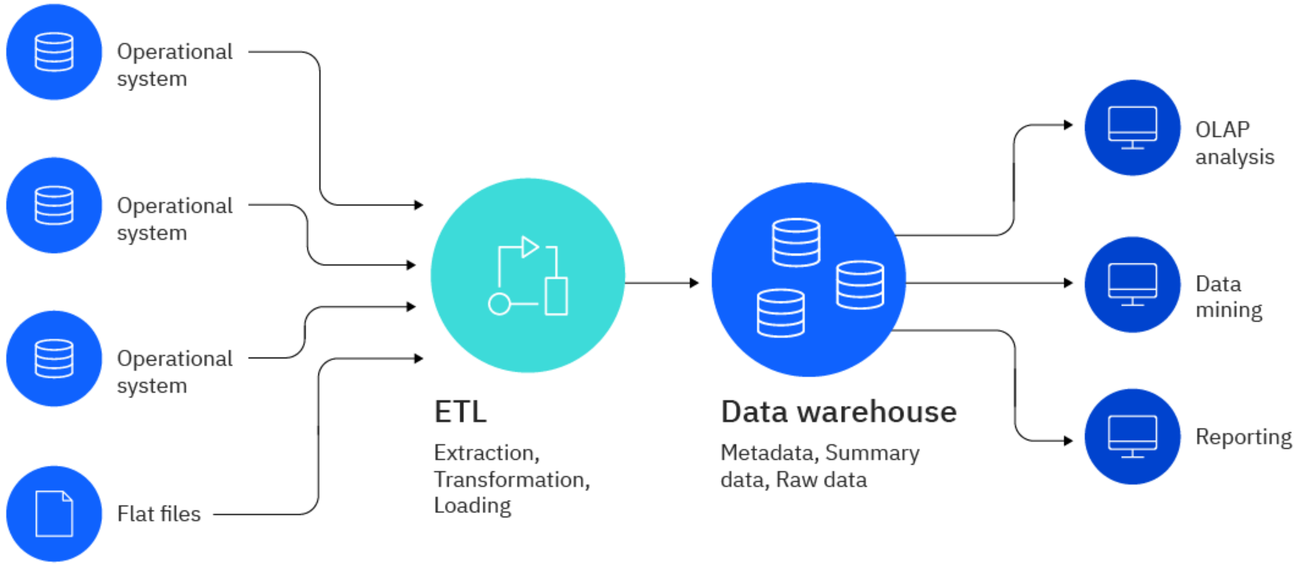


Detour: OLAP vs OLTP

Criteria	OLAP	OLTP
Purpose	OLAP helps analyze large volumes of data to support decision-making.	OLTP helps you manage and process real-time transactions.
Data source	OLAP uses historical and aggregated data from multiple sources.	OLTP uses real-time and transactional data from a single source.
Data structure	OLAP uses multidimensional (cubes) or relational databases.	OLTP uses relational databases.
Data model	OLAP uses **star schema, snowflake schema**, or other analytical models.	OLTP uses **normalized** models. (3NF)
Volume of data	OLAP has large storage requirements. Think terabytes (TB) and petabytes (PB).	OLTP has comparatively smaller storage requirements. Think gigabytes (GB).
Response time	OLAP has longer response times, typically in seconds or minutes.	OLTP has shorter response times, typically in milliseconds.
Example applications	OLAP is good for analyzing trends, predicting customer behavior, and identifying profitability.	OLTP is good for processing payments, customer data management, and order processing.

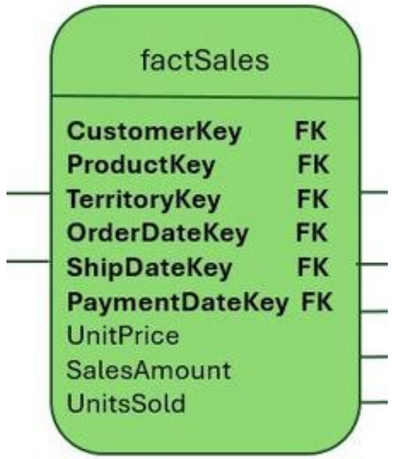
Data Warehouse - A common choice for OLAP

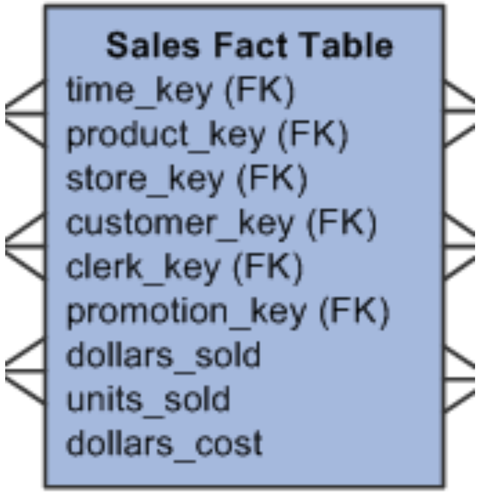
- A data warehouse is a **large, centralized repository of structured data** from various sources within an organization, specifically designed for **query and analysis** rather than transaction processing. (Redshift, Snowflake)
- It's essentially a database.
- Primary purposes:
  - Business Intelligence (BI) and reporting
  - Data analysis and decision support
  - Historical data analysis and trend identification
  - Performance metrics tracking
- **Focus on two structures: star schema and snowflake schema**
- Think of a library. You can have many books at different places (sources, databases), but also a centralized library that holds all books

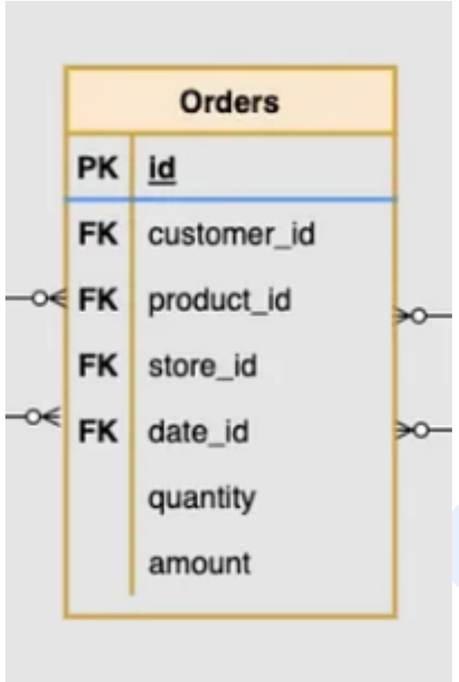


Fact Table and Dimension Table

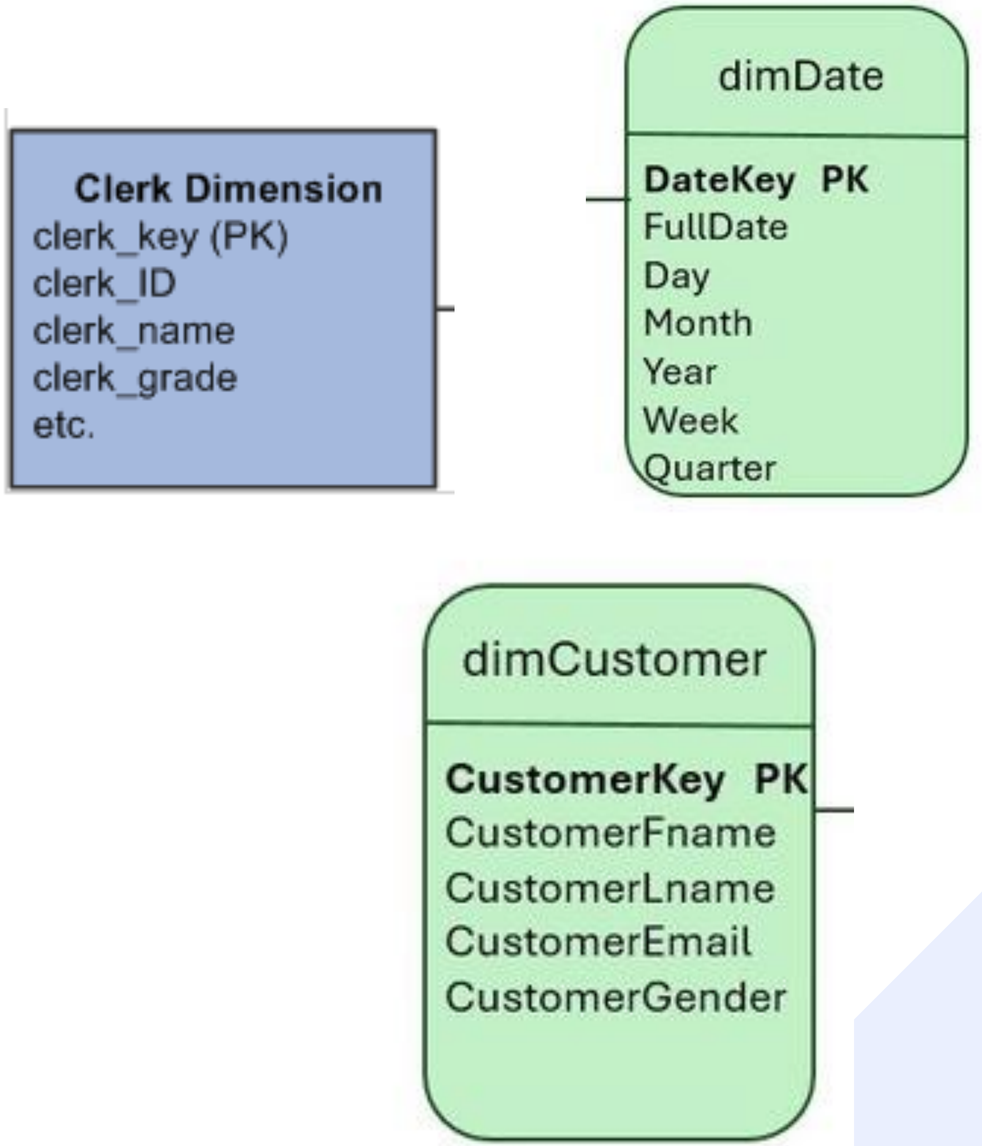
- **Fact Tables:**
  - The primary tables that store **measurable, quantitative data** about business events
  - **Contain foreign keys** to dimension tables and **numerical measures**
  - Examples of measures: sales amount, quantity sold, revenue, temperature readings
  - Typically very large with many rows, as they store transaction-level detail
  - Usually normalized and contain minimal redundancy

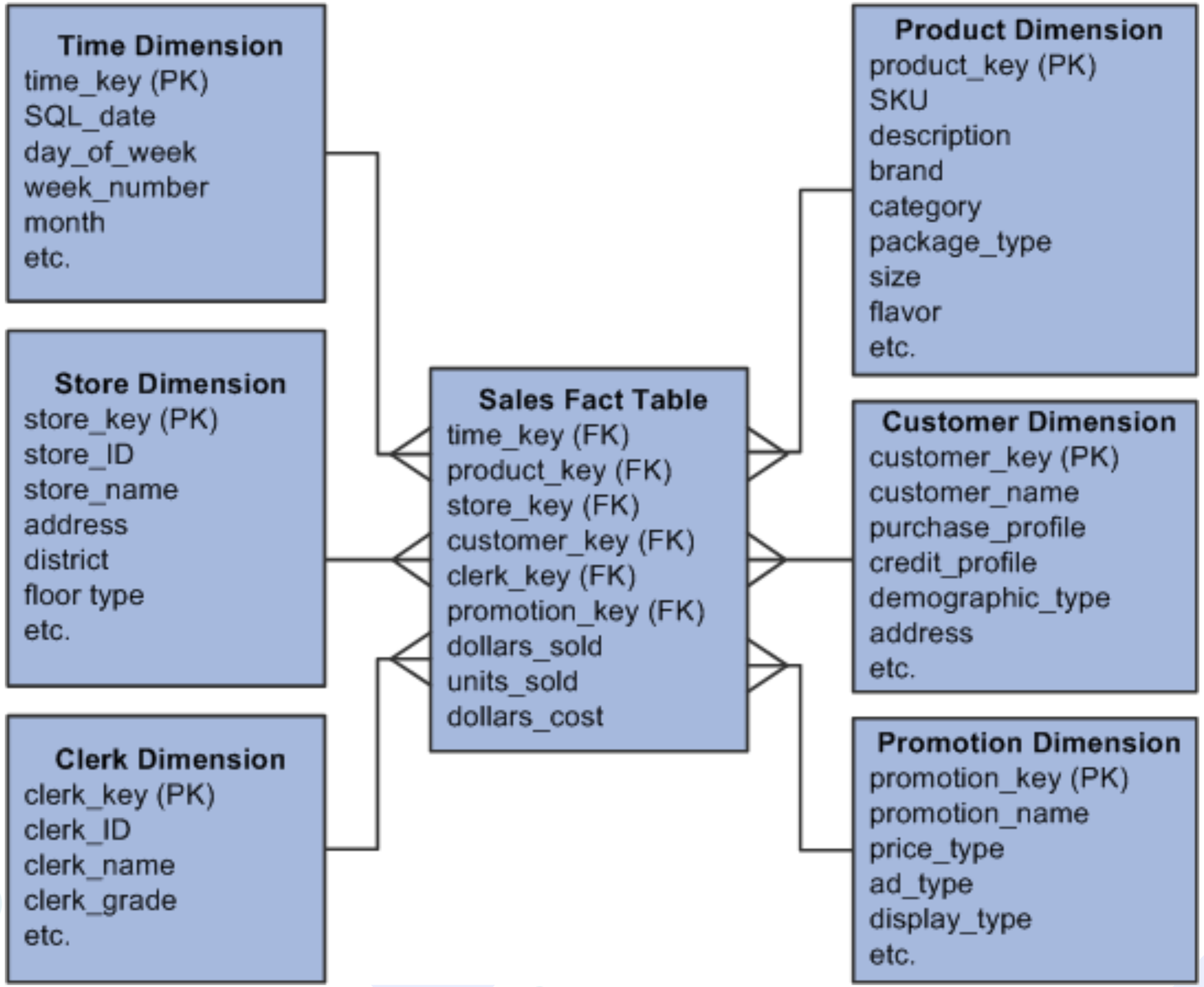




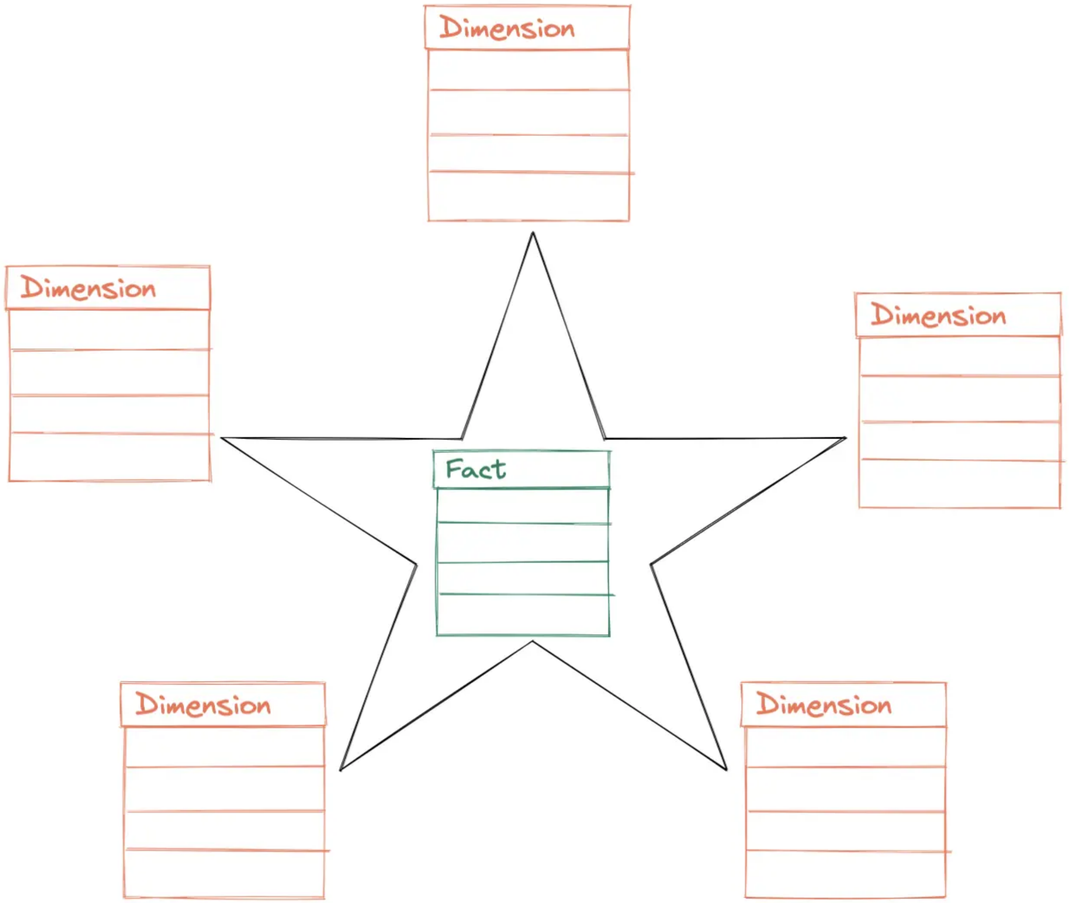


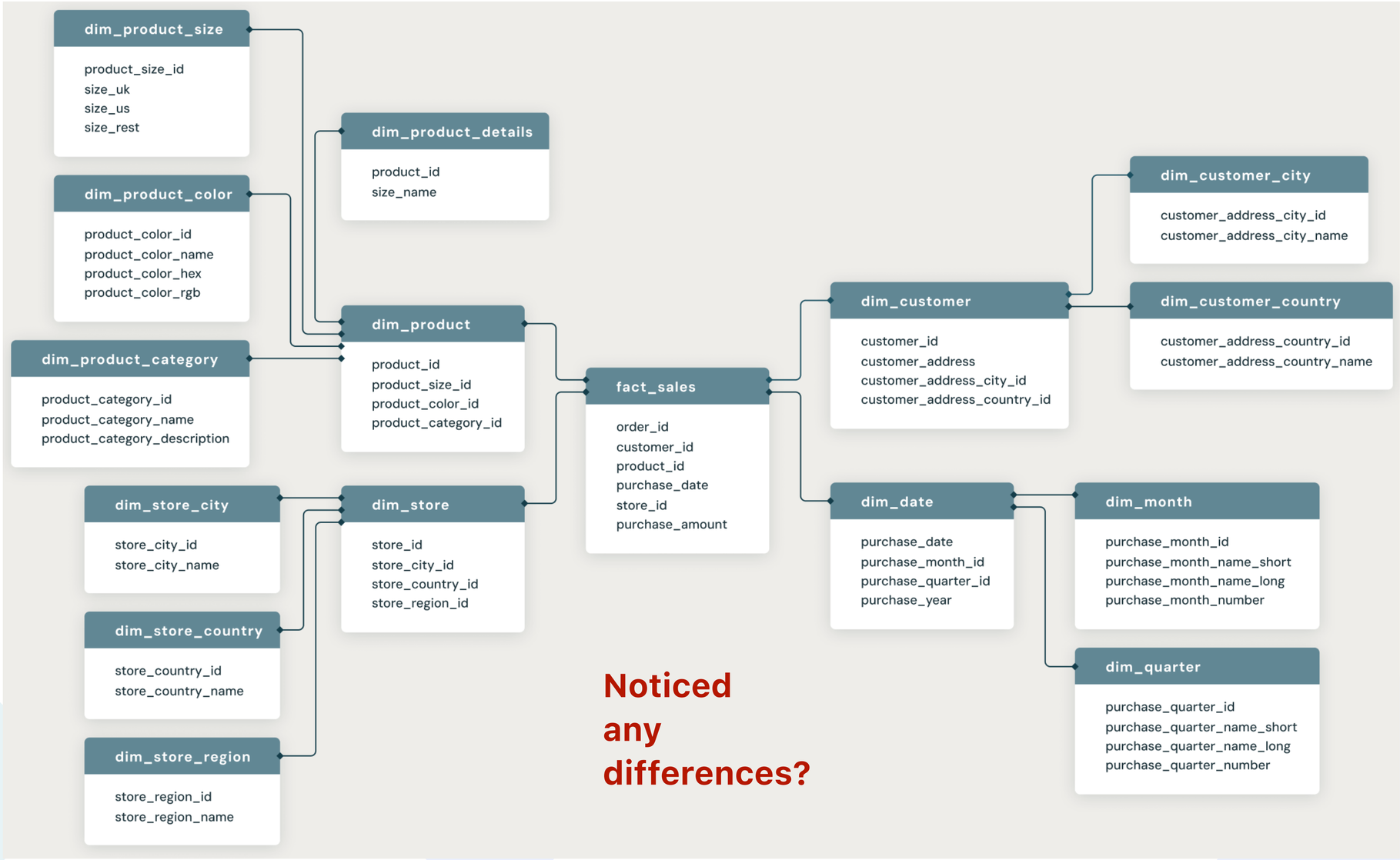
- **Dimension Tables:**
  - These contain **descriptive attributes** used to **analyze the measures in fact tables**
  - They provide the "who, what, where, when, why and how" context
  - Usually **denormalized** and contain many columns but fewer rows than fact tables
  - Used for filtering, grouping, and labeling in reports




Time Dimension
time_key (PK)
SQL_date
day_of_week
week_number
month
etc.
Store Dimension
store_key (PK)
store_ID
store_name
address
district
floor type
etc.
Clerk Dimension
clerk_key (PK)
clerk_ID
clerk_name
clerk_grade
etc.
Sales Fact Table
time_key (FK)
product_key (FK)
store_key (FK)
customer_key (FK)
clerk_key (FK)
promotion_key (FK)
dollars_sold
units_sold
dollars_cost
Product Dimension
product_key (PK)
SKU
description
brand
category
package_type
size
flavor
etc.
Customer Dimension
customer_key (PK)
customer_name
purchase_profile
credit_profile
demographic_type
address
etc.
Promotion Dimension
promotion_key (PK)
promotion_name
price_type
ad_type
display_type
etc.

Some commonly used dimension tables




dim_product_size
product_size_id
size_uk
size_us
size_rest
dim_product_details
product_id
size_name
dim_product_color
product_color_id
product_color_name
product_color_hex
product_color_rgb
dim_product
product_id
product_size_id
product_color_id
product_category_id
dim_product_category
product_category_id
product_category_name
product_category_description
dim_customer_city
customer_address_city_id
customer_address_city_name
dim_customer
customer_id
customer_address
customer_address_city_id
customer_address_country_id
dim_customer_country
customer_address_country_id
customer_address_country_name
fact_sales
order_id
customer_id
product_id
purchase_date
store_id
purchase_amount
dim_store_city
store_city_id
store_city_name
dim_store
store_id
store_city_id
store_country_id
store_region_id
dim_store_country
store_country_id
store_country_name
dim_store_region
store_region_id
store_region_name
dim_date
purchase_date
purchase_month_id
purchase_quarter_id
purchase_year
dim_month
purchase_month_id
purchase_month_name_short
purchase_month_name_long
purchase_month_number
dim_quarter
purchase_quarter_id
purchase_quarter_name_short
purchase_quarter_name_long
purchase_quarter_number
Noticed any differences?

Snowflake Schema vs Star Schema

- **Star Schema:**
  - One central Fact table, Connected directly to denormalized Dimension tables
  - Dimensions are wide and flat -> have redundant data
- **Pros:**
  - Very fast query performance
  - Simple design, easy to understand
  - Great for report and BI tools
- **Cons:**
  - Lower data Quality

- **Snowflake Schema:**
  - Also has a central Fact table, But Dimension tables are normalized into multiple smaller related tables
  - Remove redundancy
- **Pros**:
  - Less storage used
  - Better data quality and consistency
- **Cons**:
  - Slow queries(more join)
  - More complex design

OLTP - a concrete example

- Suppose we are building data storage solution for an e-commerce system. We need some efficient way to store transaction info whenever a customer places a order.

- If we use a **normalized database**: 3 quick updates, completes in milliseconds

- If we use a **DWH** instead, a complex join across all dim tables are required. Could take seconds, blocking other transactions.

```
--With regular OLTP (Normalized Databases):
INSERT INTO Orders (OrderID, CustomerID, Date) VALUES (1, 100, '2024-02-06');
INSERT INTO OrderItems (OrderID, ProductID, Quantity) VALUES (1, 201, 2);
UPDATE Inventory SET Stock = Stock - 2 WHERE ProductID = 201;



--With a Data Warehouse (DWH):
UPDATE FactSales
JOIN DimCustomer ON ...
JOIN DimProduct ON ...
JOIN DimDate ON ...
...
SET ...
```

Step 3: Requirement Refine

- Review and revise the conceptual model, ensure alignment with business goals, and address any ambiguity or inconsistencies in the requirements.

- Might not happened, depending on the development methodology (i.e. Waterfall)

Step 4: Logical Data Modeling (+ normalization)

- Translate conceptual model into a more detailed one.
  - Define table, columns, relationships in a more detailed manner.
  - Focus on reporting data in a way that supports efficient query retrieval.
  - Normalization (**not required for OLAP**)
- Thing to do/Activities:
  - Identify PK, FK
  - Normalize the data structure
  - Develop the logical relationship between entities
  - Define datatypes and attributes. Add constraints
- Normalization goal: Ensure data integrity and eliminate redundancy
  - Break entities into **well-structured relations** (e.g. 1NF, 2NF, 3NF).
  - Eliminate data anomalies:
    - Update anomalies(same data stored in multiple places)
    - Insert/ delete anomalies
  - Verify the logical model follows normalization rules.

Slowly Changing Dimensions (SCDs)

- Slowly changing dimensions are simply a dimension that stores and manages data **over time** in a data warehouse.
- SCDs are considered and implemented as one of the most critical ETL/ELT tasks in tracking the **history** of dimension records.
- For example, if we have a fact and dimension tables, the fact table is linked to the dimension tables with the help of foreign keys. If any of the dimension table data changes, we have to keep track of the data changes for reporting purposes.
- Type 1, 2, and 3 are the most commonly used types.

Type 0 SCD

- **Simple and no special action is required.**
- This type is pretty simple and doesn't require any special action since type 0-dimensional table has the static data, the values remain the **same forever**.

Example – Date dimension table (static data)

Date	day	week_in_a_month	week_in_a_year
2021-01-18	Monday	4th week	4th week
2021-02-18	Thursday	3rd week	8th week

Type 1 SCD

- **Insert and overwrite (upsert).**
- In this type, all the brand new records will get inserted and any change to the existing record will overwrite the old value with a new one. No history data is maintained in this type.

Before the change:

Emp_id	Emp_name	state
1111	Tom	MI
1212	Adam	MN

After the change:

Emp_id	Emp_name	state	
1111	Tom	AZ	Exisiting record, so overwritten the old value
1212	Adam	MN	
1818	Frank	NY	New record, so inserted

Type 2 SCD

- **Insert a changed record.**
- In this type, insert all brand new records and all changed records will terminate with the end_date and set the current_flag_status to false. Create a new record for changed data values with open end_date and set the current_flag_status to true.
- Each and every record has the effective date, end date, and flag status fields to identify in which time period the record was active.

Before the change:

Emp_id	Emp_name	state	Effective_date	end_date	current_flag_status
1111	Tom	MI	2020-10-08	9999-12-31	True
1212	Adam	MN	2020-10-08	9999-12-31	True

After the change:

Emp_id	Emp_name	state	Effective_date	end_date	current_flag_status
1111	Tom	MI	2020-10-08	2021-10-09	False
1111	Tom	AZ	2020-10-09	9999-12-31	True
1212	Adam	MN	2020-10-08	9999-12-31	True

Type 3 SCD

- **Store the new and old values in current and previous fields.**
- In this method, we usually have current and previous value fields. The new value is ingested into the current field and the old value is ingested into the previous field.
- A third column might be added as “indicator”.

Before the change:

Emp_id	Emp_name	current_state	Previous_state
1111	Tom	MI	MI
1212	Adam	MN	MN

After the change:

Emp_id	Emp_name	current_state	Previous_state
1111	Tom	AZ	MI
1212	Adam	MN	MN

Customer Key	Name	Original_State	Current_State	Effective_Date
1001	Christina	Illinois	California	2020-01-15

Implementing a SCD

- In Snowflake, we use **task** and **stream** to handle updates.
- In SQL databases (MySQL, SQL Server, PostgresSQL ...), use **triggers** and **stored procedures**.

More details after SQL lectures.

```
// Step1   ===> root task
create or replace task emp_src_truncate
warehouse=compute_wh
schedule='1440 minute'     // one day
as
truncate table emp_source;

// Step2 ===> child_task1
create or replace task emp_file_to_src_table
warehouse=compute_wh
after emp_src_truncate
as
copy into emp_source
from @emp_data_stage
file_format=(type=CSV, skip_header=1);

// Step3 ===> child_task2
create or replace task emp_scd1
warehouse=compute_wh
after emp_file_to_src_table
as
merge into emp_stage_scd1 emp
using (select * from emp_source) src
on emp.emp_id = src.emp_id
when matched and (emp.emp_name!=src.emp_name or emp.emp_state!=src.emp_state)
then
update set emp.emp_name=src.emp_name,
emp.emp_state=src.emp_state
when not matched
then
insert(emp_id,emp_name,emp_state) values(src.emp_id,src.emp_name,src.emp_state);

//Step4 ===> child_task4
Create or replace task src_file_remove
```

Spoiler: I did not check this code

Choosing the “Right” SCD

- Choosing the appropriate SCD type depends on the specific requirements of the data analysis you're conducting.
  - if you're analyzing sales data and need to know the product category at the time of each sale, a Type 2 SCD might be appropriate.
  - If you're tracking employee promotions, a Type 3 SCD can be better.
- Implementing Slowly Changing Dimensions effectively requires a good understanding of your **data and business needs**. Properly managing dimension changes ensures that your data warehouse remains accurate and reliable over time, allowing for meaningful analysis and reporting.

Date Dimension

- A date dimension contains a continuous range of dates that cover the entire **date period required** for the analysis.
- It includes columns that will allow a user to filter the data by almost any date logic.
- It can include the day of the week, workdays, weekends, quarters, months, years, fiscal years, or seasons.

Date	Year	Quarter	QuarterID	MonthNum	MonthID	MonthName	DayOfMonth	DayofWeek	DayName
2021-01-01	2021	1	2021Q1	1	202101	Jan	1	5	Friday
2021-01-02	2021	1	2021Q1	1	202101	Jan	2	6	Saturday
2021-01-03	2021	1	2021Q1	1	202101	Jan	3	0	Sunday
...									
2021-12-27	2021	4	2021Q4	12	202112	Dec	27	1	Monday
2021-12-28	2021	4	2021Q4	12	202112	Dec	28	2	Tuesday
2021-12-29	2021	4	2021Q4	12	202112	Dec	29	3	Wednesday
2021-12-30	2021	4	2021Q4	12	202112	Dec	30	4	Thursday
2021-12-31	2021	4	2021Q4	12	202112	Dec	31	5	Friday

Results | Messages

	TimeId	Time	Hour	Minute	MilitaryHour	MilitaryMinute	AMPM	DayPartEN	DayPartNL	HourFromTo12	HourFromTo24	Notation12	Notation24
1	1	00:00:00	1	1	0	0	AM	Night	Nacht	12:00AM - 1:00AM	00:00 - 01:00	12:00AM	00:00
2	2	00:01:00	1	2	0	1	AM	Night	Nacht	12:00AM - 1:00AM	00:00 - 01:00	12:01AM	00:01
3	3	00:02:00	1	3	0	2	AM	Night	Nacht	12:00AM - 1:00AM	00:00 - 01:00	12:02AM	00:02
4	4	00:03:00	1	4	0	3	AM	Night	Nacht	12:00AM - 1:00AM	00:00 - 01:00	12:03AM	00:03
5	5	00:04:00	1	5	0	4	AM	Night	Nacht	12:00AM - 1:00AM	00:00 - 01:00	12:04AM	00:04
6	6	00:05:00	1	6	0	5	AM	Night	Nacht	12:00AM - 1:00AM	00:00 - 01:00	12:05AM	00:05
7	7	00:06:00	1	7	0	6	AM	Night	Nacht	12:00AM - 1:00AM	00:00 - 01:00	12:06AM	00:06
8	8	00:07:00	1	8	0	7	AM	Night	Nacht	12:00AM - 1:00AM	00:00 - 01:00	12:07AM	00:07
9	9	00:08:00	1	9	0	8	AM	Night	Nacht	12:00AM - 1:00AM	00:00 - 01:00	12:08AM	00:08
10	10	00:09:00	1	10	0	9	AM	Night	Nacht	12:00AM - 1:00AM	00:00 - 01:00	12:09AM	00:09
11	11	00:10:00	1	11	0	10	AM	Night	Nacht	12:00AM - 1:00AM	00:00 - 01:00	12:10AM	00:10

Junk Dimension

- Indicator fields (Y/N) are very frequent, necessary to keep in the fact tables.
- If keep all those indicator fields in the fact table:
  - Need to build many small dimension tables
  - amount of information stored in the fact table also increases tremendously
- **Junk dimension** is the way to solve this problem.
  - In a junk dimension, we combine these indicator fields into a **single dimension (Only one needed)**.
  - the number of fields in the fact table, as well as the size of the fact table, can be decreased.
  - Each row in this table is a unique combination of all the indicator values.
- A junk dimension is a **single dimension** in a data warehouse that combines multiple **low-cardinality** attributes and indicators. Junk dimensions are **used to improve query efficiency** and make data models easier to use.

3 Indicators

3 Dims

FACT_TABLE

CUSTOMER_ID
PRODUCT_CD
TXN_ID
STORE_ID
TXN_CODE
COUPON_IND
PREPAY_IND
TXN_AMT

combine them into a single junk dimension

FACT_TABLE

CUSTOMER_ID
PRODUCT_CD
TXN_ID
STORE_ID
JUNK_ID
TXN_AMT

1 Dim

- 3 possible values for the TXN_CODE field
- 2 possible values for the COUPON_IND field
- 2 possible values for the PREPAY_IND field.
- This results in a total of 3 x 2 x 2 = 12 rows for the junk dimension table.

DIM_JUNK

JUNK_ID	TXN_CODE	COUPON_IND	PREPAY_IND
1	1	Y	Y
2	2	Y	Y
3	3	Y	Y
4	1	Y	N
5	2	Y	N
6	3	Y	N
7	1	N	Y
8	2	N	Y
9	3	N	Y
10	1	N	N
11	2	N	N
12	3	N	N

Step 5: Physical Data Modeling

- Turns your logical data model into the real implementation and chosen RDBMS. The physical database is created.
  - Represent the actual implementation of the DB
  - Include performance details such as table structure, indexes, constraints and storage considerations
  - Decision about optimization techs.(Normalize or denormalize table depending on use age).

- Things to do in this phase:
  - Define physical storage structure (table)
  - Specify the datatypes and constraints based on RDBMS
  - Create db objects (with SQL DDL)
    - DB objects:
      - Table
      - Views(saved queries)
      - Stored Procedures(predefined scripts)
      - User-defined functions
      - Triggers(automated actions on data changes)
      - Indexes (speed up queries)
  - Populate the DB(load initial data)
  - Optimize the DB(monitor, add/ remove index)

Step 6: Maintenance and evolution

- Update data models to align with the business needs.

How Does SQL Database Maintenance Work?

Index reorganization to keep fragmented indexes below 20 to 30%

Index rebuilding, either online or offline, to address fragmentation

Updating statistics, both index and column, to help develop strong query execution plans

Integrity and consistency checks customized to suit your specific needs

Repair and cleanup tasks like backing up database and transaction logs

